CLB HOLDINGS BERHAD
No 38, Jalan Riang 21, Kawasan Perindustrian Taman Gembira,
81200 Johor Bahru, Johor, Malaysia.
Tel: +607-338 3636
Email: sales@clbgroups.com

API Documentation

Version : 1.0.0

Last Updated : 9th March 2026

Language : JavaScript/TypeScript (Node.js)

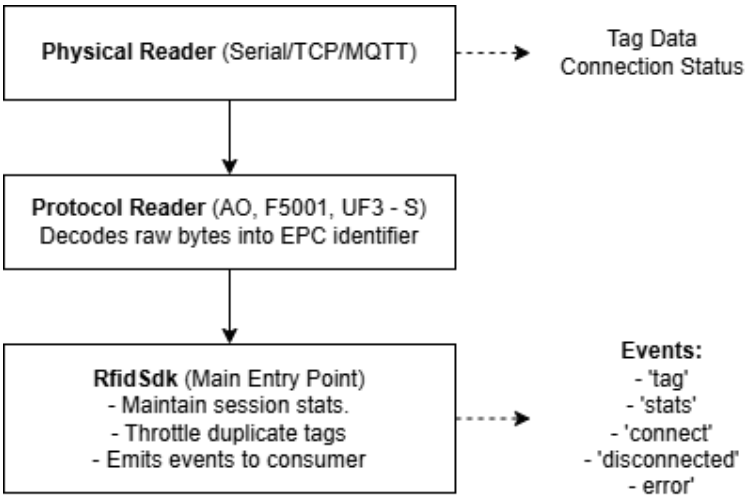
1 Overview

The Evolve RFID SDK provides a unified interface for connecting to RFID readers through multiple transport protocol (Serial, TCP/IP, MQTT) and managing tag reading operations. The SDK emits raw tag data with minimal formatting – allowing complete flexibility.

1.1 Key Design Principle

- **Transport Abstraction:** Single API works with Serial, TCP/IP, and MQTT readers.
- **Raw Data Emission:** SDK provides unformatted data; consumers handle presentation
- **In – memory session stats:** SDK maintains only session level statistics (total reads, unique tags).
- **Event – Driven Architecture:** All operations use Node.js EventEmitter pattern.
- **Automatic Throttling:** Duplicate tags are automatically suppressed within 500ms windows.
- **Structured Error Handling:** All error follow format '[HH:MM:SS][ERROR][CODE] – Message'

1.2 Event – Driven Architecture



2 Complete Event Reference

The application can register for one or more events, to be notified when it occurs. There are several types of events.

Items	Event	Descriptions
Starting and Stopping scans	*'start()'/ 'startScans()'*	Begin scanning for RFID tags
	'stop() / *'stopScan()'*	Stop scanning for RFID tags
Tag Read Events	*'tag'*	Emitted when a tag is read, passes validation, and is not throttled.
Connection Event	*'connected'*	Reader connection establish
	'disconnected'	Reader disconnected.
Statistics Events	*'stats'*	Emitted whenever statistics are updated or reset.
Error Event	*'error'*	Emitted on connection failures, protocol errors, or other

		exceptions.

2.1 Summary of Main APIs

Category	API	Purpose
Initialization	*'new RfidSdk()'*	Create SDK instance
Event Management	*'on(event, callback)'*	Register event listener
	'off(event, callback)'	Remove event listener
	'removeListener(event, callback)'	Remove listener (alias)
Connections	*'connectTcp(host, port)'*	TCP/IP reader connection
	'connectSerial(path, baud, protocol)'	Serial reader connection
	'connectMqtt(brokerURL, topic, options)'	MQTT broker connection
	'disconnect()'	Close connection gracefully
Scanning	*'start()' / 'startScan()*	Begin tag scanning
	'stop()' / 'stopScan()'	Stop tag scanning
Events	*'on('tag', …)'*	New tag detected (raw data)
	'on(stats', …)'	Statistics Updated
	'on('connected', …)'	Reader connected
	'on('disconnected', …)'	Reader disconnected
	'on('error', …)'	Error occurred (structured)
Statistics	*'getCumulativeStats()'*	Get session stats (total, unique)

	'resetCumulativeStats()'	Reset session stats to 0
Configuration	*'configure(settings)'*	Apply to reader – specific settings
	'publish(tag, topic)'	Publish to MQTT (MQTT only)

3 SDK Initialization & Connections

3.1 Class Entry Point

Class: *'rfidsdk'* – Main entry point for the RFID SDK. Initialize this class to establish connections and manage reader operations.

```
const { RfidSdk } = require('@evolve/sdk');
const sdk = new RfidSdk();
```

Description: Create a new SDK instance. The SDK maintains an internal event emitter and session statistics. Only one reader can be connected at a time; connecting to a new reader automatically disconnects the previous one. Session statistics (total reads and unique tags) are maintained in memory only and reset when the SDK is reinitialized.

Example: Basic Initialization

```
const { RfidSdk, RfidSdkError, ERROR_CODES } = require('@evolve/sdk');


// Create SDK instance
const sdk = new RfidSdk();


// Listen for tag events
sdk.on('tag', (rawTagData) => {
  console.log('Tag found:', rawTagData.epc);  // e.g., "DEADBEEF12345678"
  console.log('RSSI:', rawTagData.rssi);       // e.g., -65 dBm
  console.log('Timestamp:', rawTagData.timestamp); // Unix milliseconds
```

```
});

// Listen for statistics updates
sdk.on('stats', (stats) => {
  console.log(`Total reads: ${stats.total}, Unique tags: ${stats.unique}`);
});

// Listen for connection events
sdk.on('connected', () => {
  console.log('Reader connected and ready');
});

sdk.on('disconnected', () => {
  console.log('Reader disconnected');
});

// Listen for structured error events
sdk.on('error', (errorObj) => {
  // errorObj = { code, message, timestamp, recoverable, formatted, details }
  console.error(`[ERROR] ${errorObj.formatted}`);
  if (errorObj.recoverable) {
    console.log('Error is recoverable - system may auto-retry');
  }
});
```

3.2 RFID Reader Connection APIs

The SDK supports three transport protocol for connecting to RFID readers. Each connection methods handles the underlying transport layer details while presenting a unified interface

3.2.1 TCP/IP Connection

3.2.2 Serial Port Connection

Method: *'connectSerial(path, baudRate, protocol)'*

Description: Connect to an RFID reader via serial port. Opens a serial port connection to the reader. The SDK automatically initializes the selected protocol parser and configures serial parameter (8N1). Different readers use different command protocols; specify the correct protocol for the hardware.

```
async connectSerial(
  path: string,
  baudRate: number,
  protocol: 'UF3-S' | 'F5001' | 'A0'
): Promise<void>
```

Parameters:

Parameter	Type	Required	Default	Descriptions
'path'	string	Yes	-	Serial port path (e.g., "COM3" on Windows, "/dev/ttyUSB0" on Linux.
'baudRate'	number	Yes	-	Serial baud rate (typically 115200 or 9600)
'protocol'	string	No	'AO'	Protocol dialect: 'AO' (standard), 'F5001' (sunray), 'UF3-S' (SEUIC)

Returns: Promise that resolves when connection is establish.

Throws: '*RfidSdkError*' with code EVRFID-SERIAL-*

Example:

```
const { RfidSdk } = require('@evolve/sdk');
const sdk = new RfidSdk();

sdk.on('connected', () => {
  console.log('Serial reader ready');
  sdk.startScan();
});

sdk.on('tag', (tag) => {
  console.log(`EPC: ${tag.epc}`);
  console.log(`RSSI: ${tag.rssi}`);
  console.log(`Timestamp: ${tag.timestamp}`);
});

sdk.on('error', (err) => {
  console.error(`[${err.code}] ${err.message}`);
});

async function connect() {
  try {
    // Connect to reader using standard A0 protocol at 115200 baud
    await sdk.connectSerial('COM3', 115200, 'A0');
    // On Linux/macOS, use /dev/ttyUSB0 or similar
    // await sdk.connectSerial('/dev/ttyUSB0', 115200, 'A0');
  } catch (error) {
    console.error('Connection failed:', error.message);
  }
```

```
}

connect();
```

3.2.3 MQTT Connection

Methods: *'connectMqtt(brokerUrl, topic, options)'*

Description: Connect the RFID reader via MQTT broker. Connects to an MQTT broker and subscribes to a topic that receives RFID tag data. This enables the distributed architecture where readers publish data to a central broker, and multiple consumers subscribe to tag events. Implements exponential backoff retry with max 3 attempts.

```
async connectMqtt(
  brokerUrl: string,
  topic: string,
  options?: MqttConnectionConfig
): Promise<boolean>
```

Parameters:

Parameter	Type	Required	Descriptions
'brokerUrl'	String	Yes	MQTT broker URL (e.g., "mqtt://broker.example.com" or "mqtts://secure-broker.com")
'topic'	String	Yes	MQTT topic to subscribe to for tag data. (e.g., "rfid/reader1/tags")
'options'	MqttConnectionConfig	No	Advanced connectionoption

Returns: Promise resolving to *'true'* on success.

Throws: '*RfidSdkError*' with code EVRFID-MQTT-*

MqttConnectionConfig Interface:

```
interface MqttConnectionConfig {
  brokerUrl: string;        // Required: mqtt://host or mqtts://host
  topic: string;            // Required: MQTT topic to subscribe
  username?: string;        // Optional: MQTT username for authentication
  password?: string;        // Optional: MQTT password for authentication
  clientId?: string;        // Optional: Custom client ID (auto-generated if omitted)
  keepalive?: number;        // Optional: Keep-alive interval in seconds (default: 30)
  reconnectPeriod?: number; // Optional: Reconnect interval in ms (default: 5000)
  connectTimeout?: number;  // Optional: Connection timeout in ms (default: 10000)
  rejectUnauthorized?: boolean; // Optional: Verify TLS cert (default: true)
  protocol?: 'mqtt' | 'mqtts' | 'tcp' | 'tls' | 'ws' | 'wss'; // Optional: Force protocol
  maxRetries?: number;      // Optional: Max retry attempts (default: 3)
  [key: string]: any;       // Additional mqtt.js IClientOptions
}
```

Example:

```
const { RfidSdk } = require('@evolve/sdk');
const sdk = new RfidSdk();

sdk.on('connected', () => {
  console.log('Connected to MQTT broker');
  sdk.start();
});
```

```
sdk.on('tag', (tag) => {
  console.log(`Received tag via MQTT: ${tag.epc}`);
});

sdk.on('error', (err) => {
  console.error(`[${err.code}] ${err.message}`);
  // Recoverable errors may trigger automatic retry
});

// Basic MQTT connection
async function connectBasic() {
  try {
    await sdk.connectMqtt(
      'mqtt://broker.example.com',
      'rfid/reader1/tags'
    );
  } catch (error) {
    console.error('MQTT connection failed:', error.message);
  }
}

// Secure MQTT with authentication and TLS
async function connectSecure() {
  try {
    await sdk.connectMqtt(
      'mqtts://secure-broker.example.com:8883',
      'rfid/reader1/tags',
```

```
      {
        username: 'rfid_user',
        password: 'secure_password',
        clientId: 'evolve-reader-1',
        keepalive: 30,
        rejectUnauthorized: true,
        connectTimeout: 15000,
        maxRetries: 5
      }
    );
  } catch (error) {
    console.error('Secure connection failed:', error.message);
  }
}

connectSecure();
```

3.3 Disconnect Method

Method: *'disconnect()'*

Description: Gracefully disconnect from the reader and release resources. Close the connection to the reader, stop any active scans, remove event listeners, and cleans up resources. This method should be called before switching readers or shutting down application. Safe to call even if no reader is currently connected.

```
async disconnect(): Promise <void>
```

Parameters: None

Returns: Promise that resolves when disconnection is complete

Example:

```
async function shutdown() {
  console.log('Shutting down RFID reader...');

  sdk.stop(); // Stop scanning first (optional but recommended)

  await sdk.disconnect(); // Gracefully disconnect

  console.log('Reader disconnected, exiting');s
  process.exit(0);
}

// Graceful shutdown on process termination
process.on('SIGINT', shutdown);   // Ctrl+C
process.on('SIGTERM', shutdown);  // Termination signal
```

4 RFID Tag Reading APIs

These methods are used to control scanning and receive tag data.

4.1 Starting and stopping scans

Method: *'start()'/ 'startScans()'*

Descriptions: Initiates tag scanning from the reader. Both *'start()'* and *'startScan()'* are functional aliases for readability. The SDK will emit 'tag' events for each unique tag detected. Duplicate tags within a throttle window (500ms by default) are supposed to reduce event flooding.

```
start(): void
startScan(): void //Alias for start()
```

Parameters: None

Returns: void (synchronous)

Behaviour:

- Logs warning if no reader is connected
- Removes previous tag listener and registers a new one
- Applies automatic throttling: same (duplicate) tag within 500ms window is suppressed.
- Emits *'tag'* event for each unique tag after throttle window
- Emits *'stats'* event after each tag update
- Session statistics are updated in real – time.

Method: *'stop()* / *'stopScan()'*

Descriptions: Clearing the throttle state ensure that when *'start()'* is called again, all tags will be treated as "fresh" (not throttled) on first detection.

Parameters: None

Returns: void (synchronous)

Behaviour:

- Removes all the active tag listener
- Clears throttle state for fresh start on the next *'start()'* call.
- No *'tag'* events are emitted after this call.
- Session statistics are preserved.

4.2 Tag Read Events

Events: *'tag'*

Event Handler Signature:

```
sdk.on('tag', (rawTagData) => {
  // rawTagData structure:
  // {
  //   epc?: string,       // Electronic Product Code (primary identifier)
  //   id?: string,        // Alternative ID (protocol-specific)
  //   timestamp: number,  // Milliseconds since epoch
```

```
  //   rssi?: number,      // Signal strength in dBm
  //   raw: Buffer,        // Full raw data buffer
  //   id_full?: string    // Full payload data for debugging
  // }
});
```

Parameters:

Parameter	Type	Descriptions
'epc'	String	Primary tag identifier (typically 14 hex chars = 7 bytes). Extracted cleanly from protocol data.
'id'	String	Alternative ID field used by some protocols. Fallback if 'epc' unavailable.
'timestamp'	Number	Unix Epoch timestamp in milliseconds when tag was detected.
'rssi'	Number	Received Signal Strength Indicator in dBm. Range -50 (strong) to -90 (Weak). May be undefined if reader does not support RSSI
'raw'	Buffer	Complete raw binary from reader. For protocol analysis and debugging.
'id_full'	String	Full payload string including protocol headers. Used for debugging protocol issues.

Throttling:

- Same (duplicate) EPC within 500ms is automatically suppressed.
- Different EPC emit freely (no rate limiting)
- Throttle state is cleared when `'stop()'` is called.

Example:

```
const { RfidSdk } = require('@evolve/sdk');
const sdk = new RfidSdk();
```

```
// Simple tag listener
sdk.on('tag', (tag) => {
  console.log('Tag detected:', tag.epc);
  console.log('Signal strength:', tag.rssi, 'dBm');
});

// Filter tags by signal strength
sdk.on('tag', (tag) => {
  const signalStrength = tag.rssi || -90;

  if (signalStrength > -70) {
    console.log('Strong signal:', tag.epc);
  } else if (signalStrength > -80) {
    console.log('Moderate signal:', tag.epc);
  } else {
    console.log('Weak signal:', tag.epc);
  }
});

// Track tag history with timestamps
const tagHistory = new Map();

sdk.on('tag', (tag) => {
  // Guard against undefined EPC
  const identifier = tag.epc || tag.id;
  if (!identifier || identifier === 'UNKNOWN') return;
```

```
  if (tagHistory.has(identifier)) {
    const lastSeen = tagHistory.get(identifier);
    const timeSinceLastSeen = tag.timestamp - lastSeen;
    console.log(`Tag ${identifier} seen again after ${timeSinceLastSeen}ms`);
  } else {
    console.log(`New tag detected: ${identifier}`);
  }

  tagHistory.set(identifier, tag.timestamp);
});

// Validate and process tag data
sdk.on('tag', async (tag) => {
  const identifier = tag.epc || tag.id;

  // Validate format
  if (!identifier || identifier === 'UNKNOWN' || identifier === 'ERROR') {
    console.warn('Invalid tag identifier:', tag);
    return;
  }

  // Process valid tag
  try {
    console.log(`Processing tag: ${identifier}`);
    // Insert into database, forward to API, etc.
  } catch (error) {
    console.error('Tag processing error:', error);
```

```
  }
});
```

4.3 Session Statistics API

The SDK maintains in – memory session statistics (total tag reads and unique tag count) for current session. Statistics are NOT persistent and reset when the SDK is reinitialized.

4.3.1.1 Get & Reset Statistics

Methods: *'getCumulativeStats()'*

Descriptions: Returns cumulative counter for the current session. These statistics start at zero when the SDK is initialized and are updated in real – time as tags are detected. Only valid tags (with non – empty, non – UNKNWON EPC) are counted.

```
getCumulativeStats(): { total: number; unique: number }
```

Returns: Object with:

- *'total'*: Total number of valid tags read in current session
- *'unique'*: Count of unique EPC identifier encountered.

Method: *'resetCumulativeStats()'*

Descriptions: Reset the session counters to zero. Emits a *'stats'* event to notify all listeners of the reset.

Parameters: None

Returns: void (synchronous)

Behaviour:

- Clear total count and unique tag to 0.
- Emits *'stats'* event with reset values
- Does NOT affect historical data in database or persistent storage

4.3.1.2 Statistics Event

Event: *'stats'*

Descriptions: Emitted whenever statistics are updated or reset.

```
sdk.on('stats', (stats) => {
  // stats = { total: number, unique: number }
  console.log(`Total reads: ${stats.total}, Unique tags: ${stats.unique}`);
});
```

Emission Triggers:

- After each valid tag is detected and processed.
- When `'resetCumulativeStats()'` is called.

Example:

```
const { RfidSdk } = require('@evolve/sdk');
const sdk = new RfidSdk();

let scanStartTime = null;

sdk.on('connected', () => {
  scanStartTime = Date.now();
  sdk.start();
});

sdk.on('stats', (stats) => {
  const elapsedSeconds = (Date.now() - scanStartTime) / 1000;
  const readsPerSecond = (stats.total / elapsedSeconds).toFixed(2);

  console.log(`
    ================================
    Session Statistics:
    - Total reads: ${stats.total}
    - Unique tags: ${stats.unique}
```

```
    - Elapsed time: ${elapsedSeconds.toFixed(1)}s
    - Rate: ${readsPerSecond} tags/sec
    ================================
  `);
});

// Manual reset every 60 seconds for per-minute statistics
setInterval(() => {
  console.log('\n[Hourly statistics reset]');
  sdk.resetCumulativeStats();
  scanStartTime = Date.now();
}, 60000);

// Query at any time
function getReport() {
  const stats = sdk.getCumulativeStats();
  return {
    totalReads: stats.total,
    uniqueTags: stats.unique,
    reportTime: new Date().toISOString()
  };
}

// Export stats every 5 seconds
setInterval(() => {
  const report = getReport();
  console.log('[Report]', JSON.stringify(report));
```

```
  }, 5000);
```

5 Data Handling APIs

5.1 Raw Tag Data Structure

All tag events contain the following data structure:

```
interface TagData {
  epc: string;           // 6-7 byte hex identifier (14 hex chars)
  rssi?: number;         // Signal strength (-50 to -90 dBm)
  timestamp: number;     // Unix timestamp in milliseconds
  raw: Buffer;           // Complete raw binary payload
  id?: string;           // Alternative ID (protocol-dependent)
  id_full?: string;      // Full payload string
}
```

5.2 MQTT Message Format

When receiving tags via MQTT, the payload format depends on the reader configuration. The SDK expects JSON or binary format containing at minimum:

```
{
  "epc": "XXXXXXXXXXXXXX",
  "rssi": -65,
  "timestamp": 1640000000000
}
```

5.3 Serial Data Frame Format

Serial readers send binary frames formatted according to their protocol:

- **AO Protocol:** *'[HEADER] [LEN] [ADDR] [CMD] [DATA…] [CHECKSUM]'*
- **F5001 Protocol:** Sunray proprietary format.
- **UF3 – S Protocol:** SEUIC command/response format

5.4 Data Processing Example

```
const { RfidSdk } = require('@evolve/sdk');
const fs = require('fs');

const sdk = new RfidSdk();
const outputFile = 'tag_readings.jsonl'; // Line-delimited JSON

// Log raw frames to file
sdk.on('rawData', (packet) => {
  const logEntry = {
    packetId: packet.id,
    timestamp: packet.timestamp,
    direction: packet.direction, // 'RX' or 'TX'
    hexData: packet.data
  };
  fs.appendFileSync(outputFile, JSON.stringify(logEntry) + '\n');
});

// Process and validate tags
const validTags = new Set();

sdk.on('tag', (tag) => {
  // Validate EPC format (should be 14 hex chars)
  const epcRegex = /^[0-9A-Fa-f]{14}$/;

  if (!epcRegex.test(tag.epc)) {
```

```
    console.warn(`Invalid EPC format: ${tag.epc}`);
    return;
  }

  // Track valid unique tags
  validTags.add(tag.epc);

  // Log to file
  fs.appendFileSync(
    outputFile,
    JSON.stringify({
      type: 'TAG',
      epc: tag.epc,
      rssi: tag.rssi,
      timestamp: tag.timestamp,
      signalQuality: tag.rssi < -70 ? 'weak' : 'good'
    }) + '\n'
  );
});

// Query processed data
function getReport() {
  return {
    uniqueTags: validTags.size,
    totalReads: sdk.getCumulativeStats().total,
    exportedTo: outputFile
  };
```

```
}
```

6 Error Handling

6.1 Structured Error Format

All SDK errors are emitted with a **structured format** for consistent error handling, logging, and monitoring.

```
[HH:MM:SS] [ERROR] [CODE] – Message
```

Example:

```
[03:24:23][ERROR][EVRFID-CONN-001] - Failed to establish connection

[11:40:21][ERROR][EVRFID-SERIAL-002] - Serial port not found or unavailable

[14:15:09][ERROR][EVRFID-TCP-003] - Connection refused by remote host
```

Error Object Structure: Each error emitted via the *'error'* event is a structured object:

```
{
  code: 'EVRFID-CONN-001',              // Unique error code
  message: 'Failed to establish connection',  // Human-readable message
  timestamp: 1773027621112,             // JavaScript timestamp (ms)
  recoverable: true,                    // Boolean: can auto-retry
   formatted:  '[03:24:23][ERROR][EVRFID-CONN-001]  -  Failed  to  establish
connection',
  details: {                            // Context-specific data
    host: '192.168.1.100',
    port: 8088,
    attempts: 3,
    lastError: 'ECONNREFUSED'
  }
}
```

6.2 Error Event

Event: 'error'

Description: Emitted on connection failures, protocol errors, or other exceptions.

```
sdk.on('error', (errorObj) => {
  // errorObj is a structured error object with code, message, timestamp, etc.
  console.error(`[ERROR] ${errorObj.formatted}`);
  console.log(`Code: ${errorObj.code}`);
  console.log(`Recoverable: ${errorObj.recoverable}`);
  console.log(`Details:`, errorObj.details);
});
```

6.2.1 Common Error Scenarios

- Connection refused MQTT

```
sdk.on('error', (errorObj) => {
  if (errorObj.code === 'EVRFID-TCP-003' || errorObj.code === 'EVRFID-MQTT-
002') {
    console.error('Reader connection refused. Is it powered on?');
    console.log('Details:', errorObj.details);
    // errorObj.details: { host, port, attempts, lastError, broker, etc. }

    // Since it's recoverable, attempt retry
    if (errorObj.recoverable) {
      console.log('Will attempt auto-reconnect...');
    }
  }
});

// Example: Manual retry with exponential backoff
async function connectWithRetry(host, port, maxRetries = 5) {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
```

```
    try {
      await sdk.connectTcp(host, port);
      console.log('Connected successfully');
      return;
    } catch (error) {
      const delay = Math.pow(2, attempt) * 1000; // 2s, 4s, 8s, 16s...
      console.log(`Attempt ${attempt} failed, retrying in ${delay}ms...`);
      await new Promise(r => setTimeout(r, delay));
    }
  }
  throw new Error('Failed to connect after ' + maxRetries + ' attempts');
}
```

- Serial Port not found

```
sdk.on('error', (errorObj) => {
  if (errorObj.code === 'EVRFID-SERIAL-002') {
    console.error('Serial port not found or unavailable');
    console.log('Attempting port:', errorObj.details.port);
    console.log('Baud rate:', errorObj.details.baudRate);

    // List available ports for user
    const { SerialPort } = require('serialport');
    SerialPort.list().then(ports => {
      console.log('Available serial ports:');
      ports.forEach(port => {
        console.log(`  - ${port.path} (${port.manufacturer || 'Unknown'})`);
      });
```

```
    });
  }
});
```

6.2.2 Error Recovery

```
class RfidReaderManager {
  constructor() {
    this.sdk = new RfidSdk();
    this.setupErrorHandlers();
  }

  setupErrorHandlers() {
    this.sdk.on('error', (errorObj) => this.handleError(errorObj));
    this.sdk.on('disconnected', () => this.handleDisconnection());
  }

  handleError(errorObj) {
    const log = {
      timestamp: new Date(errorObj.timestamp).toISOString(),
      code: errorObj.code,
      message: errorObj.message,
      recoverable: errorObj.recoverable,
      details: errorObj.details,
      formatted: errorObj.formatted
    };

    // Log to file for debugging
```

```
  fs.appendFileSync('error_log.jsonl', JSON.stringify(log) + '\n');


  // Determine recovery strategy based on error code
  if (errorObj.recoverable) {
    console.warn(`[RECOVERABLE] ${errorObj.formatted}`);
    this.attemptRecovery(errorObj.code);
  } else {
    console.error(`[FATAL] ${errorObj.formatted}`);
    this.notifyAdministrator(errorObj);
  }
}


attemptRecovery(errorCode) {
  // Handle specific error codes with targeted recovery
  switch (errorCode) {
    case 'EVRFID-CONN-001':
    case 'EVRFID-CONN-002':
    case 'EVRFID-TCP-003':
    case 'EVRFID-MQTT-002':
      // Connection errors: reconnect after delay
      console.log('Reconnecting in 3 seconds...');
      setTimeout(() => this.reconnect(), 3000);
      break;


    case 'EVRFID-READER-001':
      // Reader not responding: try a reset command
      console.log('Reader unresponsive, attempting reset...');
```

```
        this.sendReaderReset();
        break;

      case 'EVRFID-SERIAL-005':
      case 'EVRFID-DATA-005':
        // I/O and database errors: may resolve themselves
        console.log('Transient I/O error, will retry next operation');
        break;

      default:
        // Generic recovery
        console.log('Attempting generic recovery...');
        this.reconnect();
    }
  }

  async reconnect() {
    try {
      console.log('Reconnecting...');
      await this.disconnect();
      await new Promise(r => setTimeout(r, 1000));
      await this.connect();
      console.log('Reconnection successful');
    } catch (error) {
      console.error('Reconnect failed:', error);
    }
  }
```

```
  handleDisconnection() {
    console.warn('Connection lost, will retry in 5 seconds...');
    setTimeout(() => this.reconnect(), 5000);
  }

  async disconnect() {
    await this.sdk.disconnect();
  }

  async connect() {
    // Your connection logic - adjust based on your transport
    await this.sdk.connectTcp('192.168.1.100', 10001);
  }

  sendReaderReset() {
    // Protocol-specific reset command
    console.log('Sending reader reset command...');
  }

  notifyAdministrator(errorObj) {
    // Send alert for non-recoverable errors
    console.error(`ALERT: Non-recoverable error: ${errorObj.code}`);
    // Implement email/SMS/Slack notification here
  }
}
// Usage
const manager = new RfidReaderManager();
```